

UniApp【萌宠案例】分段器组件点击切换实现方案

在萌宠案例的分类展示、内容筛选等场景中，分段器（Segmented Control）是提升用户操作效率的核心组件。本方案基于uni-ui的 `uni-segmented-control` 组件，以萌宠“类型分类”和“排序筛选”双场景为例，实现点击切换时的状态同步、数据联动、加载反馈等完整逻辑，同时兼顾多端样式一致性与交互流畅性。

一、基础认知：uni-segmented-control组件核心特性

uni-ui的分段器组件支持“按钮模式”“卡片模式”两种核心样式，通过 `v-model` 实现选中状态双向绑定，`@clickItem` 事件响应点击操作，核心优势包括：

- 双向绑定选中索引，状态管理简洁，无需手动修改选中值
- 支持自定义激活色、边框色、高度等样式，适配不同设计风格
- 点击事件返回清晰的索引参数，便于关联后续业务逻辑
- 多端自适应，在小程序、App、H5端表现一致



核心属性说明：

- `v-model`：绑定当前选中的索引（Number类型），必填
- `values`：分段器选项数组（String数组或Object数组），必填
- `mode`：样式模式，可选“button”（按钮式）或“card”（卡片式），默认“button”
- `@clickItem`：点击选项时触发，回调参数为选中索引（index）

二、前置准备：环境配置与组件引入

1. 环境配置

确保已完成uni-ui组件库的安装与配置，若未配置需先执行以下步骤：

1. HBuilderX中右键项目 → 插件安装 → 搜索“uni-ui” → 安装官方组件库
2. 在项目根目录 `pages.json` 中配置easycom规则，实现组件自动按需引入：

```

1 Code block
2 {
3   "easycom": {
4     "autoscan": true,
5     "custom": {
6       "^uni-(.*)": "@uni_modules/uni-$1/uni-$1.vue"
7     }
8   },
9   "pages": [
10    {
11      "path": "pages/pet-list/pet-list",
12      "style": {
13        "navigationBarTitleText": "萌宠列表"
14      }
15    }
16  ]
17 }
18

```

三、核心场景一：萌宠类型分类切换（卡片模式）

以萌宠列表页的“类型分类”为例，使用卡片模式分段器，实现“全部、猫咪、狗狗、其他宠物”的点击切换，并联动加载对应类型的萌宠数据，包含加载状态、空数据提示等完整反馈。

1. 页面结构（pages/pet-list/pet-list.vue）

```

1 Code block
2 <template>
3   <view class="pet-list-page">
4
5     <view class="page-header">萌宠列表</view>
6
7
8     <uni-segmented-control
9       v-model="currentTypeIndex"
10      :values="typeValues"
11      mode="card"
12      @clickItem="handleTypeClick"
13      activeColor="#ff6700"
14      borderColor="#f0f0f0"
15      height="80rpx"
16      fontSize="28rpx"
17      class="type-segmented"

```

```

18     ></uni-segmented-control>
19
20
21     <uni-load-more
22         v-if="isLoading"
23         status="loading"
24         class="load-more"
25     ></uni-load-more>
26
27
28     <view class="pet-list" v-if="petList.length && !isLoading">
29         <view class="pet-item" v-for="(pet, index) in petList" :key="pet.id +
index">
30             <image :src="pet.coverImg" mode="aspectFill" class="pet-img"></image>
31             <view class="pet-info">
32                 <view class="pet-name">{{ pet.name }}</view>
33                 <view class="pet-breed">品种: {{ pet.breed }}</view>
34                 <view class="pet-like">
35                     <uni-icon type="heart" size="24rpx" color="#ff4444"></uni-icon>
36                     <span class="like-count">{{ pet.likeCount }}</span>
37                 </view>
38             </view>
39         </view>
40     </view>
41
42
43     <uni-empty
44         v-if="petList.length === 0 && !isLoading"
45         text="暂无该类型萌宠数据"
46         mode="list"
47         class="empty-tip"
48     ></uni-empty>
49 </view>
50 </template>
51

```

2. 逻辑实现（核心：点击切换与数据联动）

Code block

```

1
2 <script setup>
3 import { ref, onLoad } from 'vue'
4 import request from '../utils/request'
5
6 // 1. 分段器核心状态

```

```
7  const currentIndex = ref(0) // 当前选中的类型索引，双向绑定
8  const typeValues = ref(['全部', '猫咪', '狗狗', '其他宠物']) // 类型选项
9  const typeMap = { // 索引与后端接口类型的映射表，便于数据请求
10    0: 'all',
11    1: 'cat',
12    2: 'dog',
13    3: 'other'
14  }
15
16  // 2. 列表与加载状态
17  const petList = ref([])
18  const isLoading = ref(false)
19
20  // 3. 页面加载：初始化加载默认类型数据
21  onLoad(() => {
22    loadPetData()
23  })
24
25  // 4. 分段器点击事件：切换类型并加载数据
26  const handleClick = (index) => {
27    // 若点击的是当前选中项，无需重复加载
28    if (currentIndex.value === index) return
29    // 更新选中索引（v-model会自动同步，此处可省略，但保留便于理解）
30    currentIndex.value = index
31    // 重置列表并加载新数据
32    petList.value = []
33    loadPetData()
34  }
35
36  // 5. 核心：加载萌宠数据（根据选中类型请求）
37  const loadPetData = () => {
38    isLoading.value = true
39    // 获取当前选中类型对应的接口参数
40    const currentType = typeMap[currentIndex.value]
41    // 发起请求
42    request.get('/pet/list', { type: currentType, page: 1, pageSize: 10 })
43      .then((res) => {
44        // 筛选有效数据（避免无图、无名的无效条目）
45        const validData = res.data.filter(item => item.id && item.coverImg &&
item.name)
46        petList.value = validData
47      })
48      .catch((err) => {
49        uni.showToast({ title: err.msg || '数据加载失败', icon: 'error' })
50      })
51      .finally(() => {
52        isLoading.value = false
```

```
53     })
54   }
55 </script>
56
```

3. 样式实现（适配卡片模式视觉效果）

Code block

```
1
2  .pet-list-page {
3    background-color: #f5f5f5;
4    min-height: 100vh;
5    padding: 20rpx;
6  }
7
8  .page-header {
9    font-size: 36rpx;
10   font-weight: 500;
11   color: #333;
12   margin: 20rpx 0 30rpx;
13 }
14
15 /* 分段器样式 */
16 .type-segmented {
17   margin-bottom: 30rpx;
18   --uni-segmented-control-card-border-radius: 100rpx; // 卡片模式圆角
19   --uni-segmented-control-active-color: #ff6700; // 激活色
20   --uni-segmented-control-border-color: #f0f0f0; // 边框色
21   --uni-segmented-control-card-bg-color: #fff; // 卡片背景色
22 }
23
24 /* 加载状态 */
25 .load-more {
26   text-align: center;
27   padding: 50rpx 0;
28 }
29
30 /* 萌宠列表样式 */
31 .pet-list {
32   display: flex;
33   flex-wrap: wrap;
34   justify-content: space-between;
35   gap: 20rpx;
36 }
37
```

```
38 .pet-item {
39   width: calc(50% - 10rpx);
40   background-color: #fff;
41   border-radius: 16rpx;
42   overflow: hidden;
43   box-shadow: 0 2rpx 8rpx rgba(0, 0, 0, 0.05);
44 }
45
46 .pet-img {
47   width: 100%;
48   height: 280rpx;
49 }
50
51 .pet-info {
52   padding: 20rpx;
53 }
54
55 .pet-name {
56   font-size: 28rpx;
57   font-weight: 500;
58   margin-bottom: 8rpx;
59 }
60
61 .pet-breed {
62   font-size: 24rpx;
63   color: #666;
64   margin-bottom: 16rpx;
65 }
66
67 .pet-like {
68   display: flex;
69   align-items: center;
70   font-size: 22rpx;
71   color: #999;
72   gap: 8rpx;
73 }
74
75 /* 空数据提示 */
76 .empty-tip {
77   margin-top: 200rpx;
78 }
79
```

四、核心场景二：萌宠排序筛选切换（按钮模式）

在萌宠列表页添加“排序筛选”功能，使用按钮模式分段器，实现“推荐排序、最新上架、人气最高”的点击切换，与类型分类形成组合筛选，提升筛选灵活性。

1. 页面结构扩展（添加排序分段器）

Code block

```
1
2  <template>
3    <view class="pet-list-page">
4
5      <view class="page-header">萌宠列表</view>
6
7
8      <uni-segmented-control
9        v-model="currentTypeIndex"
10       :values="typeValues"
11       mode="card"
12       @clickItem="handleTypeClick"
13       class="type-segmented"
14     ></uni-segmented-control>
15
16
17     <uni-segmented-control
18       v-model="currentSortIndex"
19       :values="sortValues"
20       mode="button"
21       @clickItem="handleSortClick"
22       activeColor="#ff6700"
23       height="72rpx"
24       fontSize="26rpx"
25       class="sort-segmented"
26     ></uni-segmented-control>
27
28
29   </view>
30 </template>
31
```

2. 逻辑扩展（组合筛选与数据联动）

Code block

```
1
2  <script setup>
3    import { ref, onLoad } from 'vue'
```

```
4 import request from '../utils/request'
5
6 // 1. 类型分类相关（与场景——一致）
7 const currentIndex = ref(0)
8 const typeValues = ref(['全部', '猫咪', '狗狗', '其他宠物'])
9 const typeMap = { 0: 'all', 1: 'cat', 2: 'dog', 3: 'other' }
10
11 // 2. 排序筛选相关（新增）
12 const currentSortIndex = ref(0) // 排序选中索引
13 const sortValues = ref(['推荐排序', '最新上架', '人气最高']) // 排序选项
14 const sortMap = { // 索引与排序字段的映射
15   0: 'recommend',
16   1: 'newest',
17   2: 'hot'
18 }
19
20 // 3. 列表与加载状态
21 const petList = ref([])
22 const isLoading = ref(false)
23
24 // 4. 页面加载：初始化加载
25 onLoad(() => {
26   loadPetData()
27 })
28
29 // 5. 类型分段器点击
30 const handleTypeClick = (index) => {
31   if (currentIndex.value === index) return
32   currentIndex.value = index
33   loadPetData()
34 }
35
36 // 6. 排序分段器点击
37 const handleSortClick = (index) => {
38   if (currentSortIndex.value === index) return
39   currentSortIndex.value = index
40   loadPetData()
41 }
42
43 // 7. 核心：组合筛选加载数据（类型+排序）
44 const loadPetData = () => {
45   isLoading.value = true
46   // 组合筛选参数：类型 + 排序
47   const params = {
48     type: typeMap[currentIndex.value],
49     sort: sortMap[currentSortIndex.value],
50     page: 1,
```



```

51     pageSize: 10
52   }
53   // 发起请求
54   request.get('/pet/list', params)
55     .then((res) => {
56       const validData = res.data.filter(item => item.id && item.coverImg)
57       petList.value = validData
58       // 显示筛选结果提示
59       const currentType = typeValues.value[currentTypeIndex.value]
60       const currentSort = sortValues.value[currentSortIndex.value]
61       uni.showToast({ title: `已筛选: ${currentType} · ${currentSort}`, icon:
'none' })
62     })
63     .catch((err) => {
64       uni.showToast({ title: err.msg || '数据加载失败', icon: 'error' })
65     })
66     .finally(() => {
67       isLoading.value = false
68     })
69   }
70 </script>
71

```

3. 排序分段器样式

Code block

```

1
2  /* 排序分段器样式 */
3  .sort-segmented {
4    margin-bottom: 30rpx;
5    --uni-segmented-control-button-border-radius: 8rpx; // 按钮模式圆角
6    --uni-segmented-control-active-bg-color: #fff3e8; // 激活背景色
7    --uni-segmented-control-button-border-color: #ff6700; // 按钮边框色
8  }
9

```

五、进阶优化：提升切换体验的核心技巧

1. 避免重复请求与快速点击问题

用户快速点击分段器时可能导致多次重复请求，通过“加载锁”优化：

Code block

```

1
2 // 新增加载锁状态
3 const isRequesting = ref(false)
4
5 const loadPetData = () => {
6   // 若正在请求，直接返回
7   if (isRequesting.value) return
8   isRequesting.value = true
9   isLoading.value = true
10
11   const params = { /* 筛选参数 */ }
12   request.get('/pet/list', params)
13     .then((res) => { /* 数据处理 */ })
14     .catch((err) => { /* 错误处理 */ })
15     .finally(() => {
16       isLoading.value = false
17       isRequesting.value = false // 释放请求锁
18     })
19 }
20
21 // 点击事件中添加加载状态判断
22 const handleTypeClick = (index) => {
23   if (currentTypeIndex.value === index || isRequesting.value) return
24   currentTypeIndex.value = index
25   loadPetData()
26 }
27

```

2. 平滑过渡动画（可选）

为列表添加淡入淡出动画，提升切换时的视觉体验：

Code block

```

1
2 <!-- 为列表添加动画类 -->
3 <view class="pet-list" v-if="petList.length && !isLoading" :class="{ 'fade-
  in': isShowList }">
4
5 </view>
6

```

Code block

```

1
2 // 新增列表显示状态

```

```

3  const isShowList = ref(true)
4
5  const handleTypeClick = (index) => {
6    if (currentTypeIndex.value === index || isRequesting.value) return
7    currentTypeIndex.value = index
8    isShowList.value = false // 隐藏列表
9    // 延迟50ms加载数据, 让隐藏动画完成
10   setTimeout(() => {
11     loadPetData().then(() => {
12       isShowList.value = true // 数据加载完成后显示列表
13     })
14   }, 50)
15 }
16

```

Code block

```

1
2  /* 淡入动画 */
3  .fade-in {
4    animation: fadeIn 0.3s ease-in-out;
5  }
6
7  @keyframes fadeIn {
8    from { opacity: 0; transform: translateY(20rpx); }
9    to { opacity: 1; transform: translateY(0); }
10 }
11

```

3. 自定义分段器选项（含图标）

若需在分段器选项中添加图标，可将 `values` 设置为Object数组，配合slot实现：

Code block

```

1
2  <uni-segmented-control
3    v-model="currentSortIndex"
4    :values="sortValues"
5    mode="button"
6    @clickItem="handleSortClick"
7  >
8    <template #item=" {index, item} " >
9      <view class="sort-item">
10        <uni-icon :type="item.icon" size="24rpx" :color="currentSortIndex ===
            index ? '#ff6700' : '#666'"></uni-icon>

```

```
11         <span class="sort-text">{{ item.text }}</span>
12     </view>
13 </template>
14 </uni-segmented-control>
15
```

Code block

```
1
2 // 排序选项数组改为Object格式
3 const sortValues = ref([
4   { text: '推荐排序', icon: 'star' },
5   { text: '最新上架', icon: 'clock' },
6   { text: '人气最高', icon: 'fire' }
7 ])
8 // 排序映射表调整为匹配索引
9 const sortMap = { 0: 'recommend', 1: 'newest', 2: 'hot' }
10
```

六、多端适配与常见问题解决

1. 多端适配要点

- **小程序端**：确保分段器父容器无 `overflow: hidden`，避免卡片模式圆角被截断；基础库版本建议 $\geq 2.20.0$ 。
- **App端**：Android端若出现点击无响应，检查是否添加 `clickable` 属性（部分版本需手动添加）；iOS端动画流畅度无需额外优化。
- **H5端**：通过CSS变量统一分段器高度、字体大小，避免浏览器默认样式干扰；添加 `cursor: pointer` 优化鼠标悬浮体验。

2. 常见问题与解决方案

问题现象	原因分析	解决方案
分段器选中状态不同步	未使用v-model绑定索引，或手动修改索引后未触发更新	1. 必须使用v-model双向绑定；2. 避免直接操作DOM，通过修改绑定的索引值实现状态更新
点击事件无响应	1. 事件名称拼写错误（如clickitem小写）；2. 组件被	1. 确认事件名为@clickItem（首字母大写）；2. 检查z-

	其他元素遮挡；3. 加载锁逻辑异常	index层级；3. 调试加载锁状态是否正常释放
卡片模式圆角显示异常	父容器有边框或overflow:hidden，或未通过CSS变量设置圆角	1. 为分段器设置独立父容器； 2. 通过--uni-segmented-control-card-border-radius设置圆角
多端样式不一致	使用了固定px单位，或未通过组件CSS变量修改样式	1. 统一使用rpx单位；2. 优先通过组件自带CSS变量修改样式，而非!important覆盖

七、总结

本方案基于uni-ui分段器组件，为萌宠案例实现了从基础点击切换到组合筛选的完整功能，核心亮点包括：

- 1. **状态管理清晰**：通过v-model双向绑定选中索引，无需手动维护选中状态，降低逻辑复杂度。
- 2. **数据联动精准**：建立索引与接口参数的映射表，点击切换时快速匹配请求参数，确保数据精准性。
- 3. **体验优化到位**：通过加载锁、过渡动画等技巧，解决重复请求、切换生硬等问题，提升用户体验。
- 4. **多端适配稳定**：基于组件CSS变量与rpx单位，确保在小程序、App、H5端样式与交互一致。

该实现可灵活复用至萌宠案例的其他页面，如“萌宠详情页的标签切换”“个人中心的功能分类”等场景，只需调整分段器选项与数据加载逻辑即可快速适配。

（注：文档部分内容可能由 AI 生成）